Capstone

May 29, 2025

[ ]:
```
## Importing required Libraries
import os
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
from scipy import stats
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from statsmodels.tsa.arima.model import ARIMA
```

[10]:
```
## Loading Datasets into Jupyter Notebook

# Annual Average Growth in Ontario
annual_average_growth = pd.read_excel("D:\\One␣
 ↪Drive\\OneDrive\\CIMT\\Capstone\\Data\\Date Alighned - 2014 - 2021\\Average␣
 ↪annual growth.xlsx")

# Immigrants Admissions Category
immigrants_admissions_category = pd.read_excel("D:\\One␣
 ↪Drive\\OneDrive\\CIMT\\Capstone\\Data\\Date Alighned - 2014 -␣
 ↪2021\\Immigrants Admissions Category.xlsx")

# Immigrants Income in terms of dollars
immigrants_income_amounts = pd.read_excel("D:\\One␣
 ↪Drive\\OneDrive\\CIMT\\Capstone\\Data\\Date Alighned - 2014 -␣
 ↪2021\\immigrants income - Amounts.xlsx")

# Immigrants Income in terms of Proportions
immigrants_income_proportions = pd.read_excel("D:\\One␣
 ↪Drive\\OneDrive\\CIMT\\Capstone\\Data\\Date Alighned - 2014 -␣
 ↪2021\\immigrants income - Proportions.xlsx")

# Government spending per Capita over the years
```

```
government_expense_per_capita = pd.read_excel("D:\\One␣
 ↪Drive\\OneDrive\\CIMT\\Capstone\\Data\\Date Alighned - 2014 -␣
 ↪2021\\Government expense per capita.xlsx")

# Government spending by sector in cent
government_spending_per_sector = pd.read_excel("D:\\One␣
 ↪Drive\\OneDrive\\CIMT\\Capstone\\Data\\Date Alighned - 2014 -␣
 ↪2021\\government Spending by sector.xlsx")

# Social assistance cases over the years
social_assistance_case_yearly = pd.read_excel("D:\\One␣
 ↪Drive\\OneDrive\\CIMT\\Capstone\\Data\\Date Alighned - 2014 - 2021\\social␣
 ↪Assistance cases by year.xlsx")

# Total Government Spending in Billions
total_government_spending = pd.read_excel("D:\\One␣
 ↪Drive\\OneDrive\\CIMT\\Capstone\\Data\\Date Alighned - 2014 - 2021\\Total␣
 ↪govt spending - fao - ontario.xlsx")

# Total Income of Ontario Residents
total_income_residents = pd.read_excel("D:\\One␣
 ↪Drive\\OneDrive\\CIMT\\Capstone\\Data\\Date Alighned - 2014 - 2021\\Total␣
 ↪income of ontario residents - statscan\\Consolidated.xlsx")

# Consolidated Table -- Containing major relevant columns from all tables
consolidated_table = pd.read_excel("C:\\Users\\Saad␣
 ↪Ehsan\\Cleaned_Data\\Consolidated Table - From Power BI.xlsx")
```

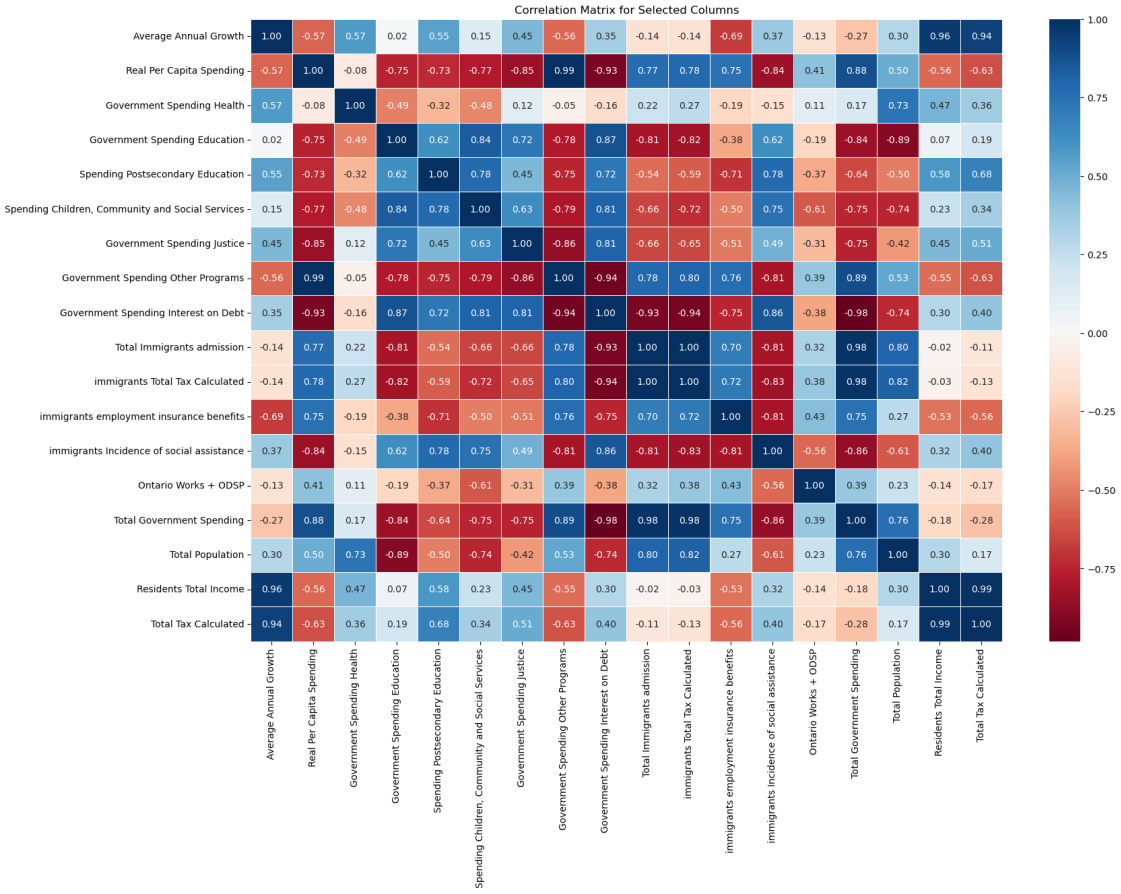
[11]:
```
## CLeaning the data set

# Dropping NaN's
immigrants_admissions_category=immigrants_admissions_category.dropna(how='all')
annual_average_growth=annual_average_growth.dropna(how='all')
immigrants_income_amounts=immigrants_income_amounts.dropna(how='all')
immigrants_income_proportions=immigrants_income_proportions.dropna(how='all')
government_expense_per_capita=government_expense_per_capita.dropna(how='all')
government_spending_per_sector=government_spending_per_sector.dropna(how='all')
social_assistance_case_yearly=social_assistance_case_yearly.dropna(how='all')
total_government_spending=total_government_spending.dropna(how='all')
total_income_residents=total_income_residents.dropna(how='all')

# Converting Columns to percentages
government_spending_per_sector.iloc[:,1:]=government_spending_per_sector.iloc[:
 ↪,1:]*100

# Round Values to two decimal places
immigrants_income_proportions=immigrants_income_proportions.round(2)
```

```
immigrants_income_amounts=immigrants_income_amounts.round(2)

# Deleting row in Social Assistance Table
social_assistance_case_yearly = social_assistance_case_yearly.drop(index=0)

# Renaming Columns
social_assistance_case_yearly.columns=["Fiscal Year" , " Ontario Works + ODSP "]

# Showing full numbers and not exponents
pd.set_option('display.float_format','{:.0f}'.format)
```

[12]:
```
## Processing and transforming the income table of immigrants to compute tax␣
 ↪obligations.
## Due to the sensitivity of the data, the estimated tax values are derived␣
 ↪using limited accessible information.

# allocating immigrants table to a new name "df"
df2 = immigrants_income_amounts
df2=pd.DataFrame(df2)
# dropping all rows except total income and header
df2=df2.iloc[:2]

# Defining Tax rate (Includes both provincial and federal, based on 2024 tax␣
 ↪rates)
tax_rate = 0.2005

# adding a new row with calculated tax values
df2.loc[len(df2)] = ["Estimated Tax Per capita"] + [round(income * tax_rate, 2)␣
 ↪for income in df2.iloc[1, 1:].values]

# Add "Total Tax Calculated" row (Estimated Tax * Total number of residents)
df2.loc[len(df2)] = ["Total Tax Calculated"] + [round(df2.iloc[2, i] * df2.
 ↪iloc[0, i], 2) for i in range(1, len(df2.columns))]

# Deleting index column and resetting index
df2 = df2.reset_index()
df2 = df2.drop(columns=['index'])

# Renaming df2 to immigrants income
immigrants_income_amounts_with_tax = df2
```

[13]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# Creating a correlation matrix
corr_matrix = consolidated_table.corr()
```

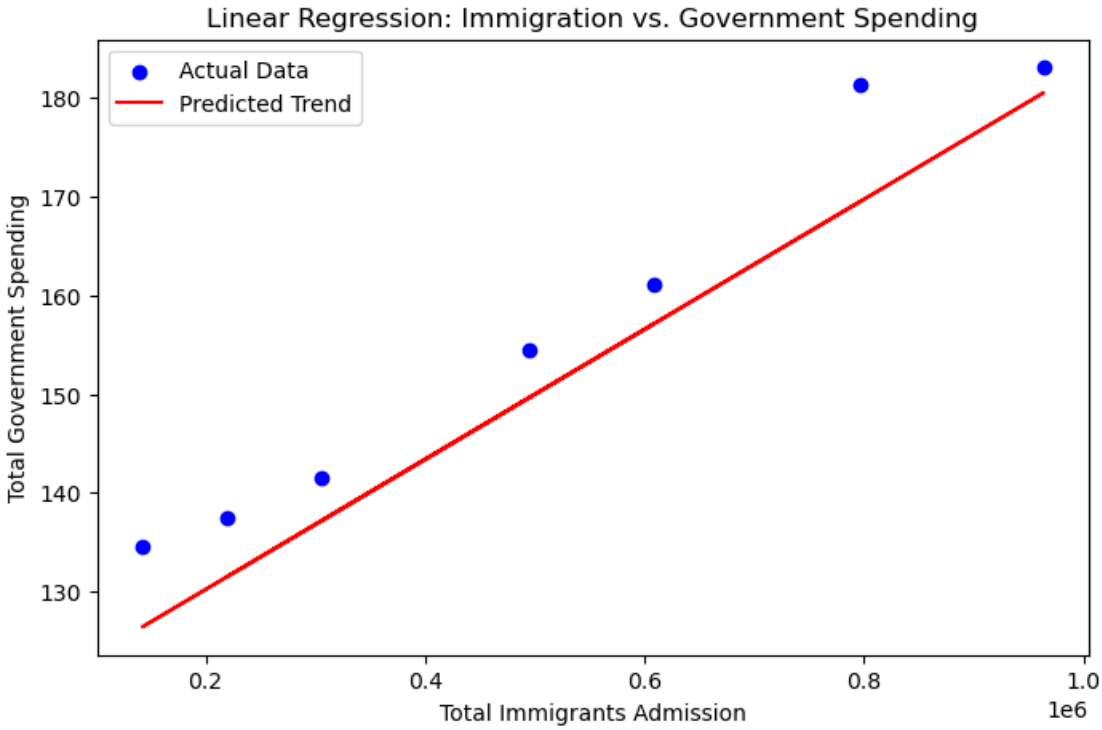
```
# Set figure size
plt.figure(figsize=(18,12))

# Create heatmap
sns.heatmap(corr_matrix, annot=True, cmap="RdBu", fmt=".2f", linewidths=0.5)

# Add title
plt.title("Correlation Matrix for Selected Columns")

# Show plot
plt.show()
```



[22]:
```
## Regression Analysis - Linear Regression: Immigration vs. Government Spending

# Standardising table for column names  - lower case letter
consolidated_table.columns = consolidated_table.columns.str.lower().str.strip()

# Defining predictor(X) and target variable (Y)
X = consolidated_table[['total immigrants admission']]
```

```
Y = consolidated_table['total government spending']

# Splitting data for training and testing ( 80 / 20 Split )
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.7,␣
 ↪random_state=42)

# Initialising model
model = LinearRegression()

# Training model
model.fit(X_train, Y_train)

# Predicting Values
Y_pred = model.predict(X_test)

# Printing regression copefficients
print(f"Intercept: {model.intercept_}")
print(f"Coefficient: {model.coef_}")

## Evaluating Model
from sklearn.metrics import mean_squared_error, r2_score

# Evaluate model accuracy
mse = mean_squared_error(Y_test, Y_pred)
r2 = r2_score(Y_test, Y_pred)

print(f"Mean Squared Error: {mse:.2f}")
print(f"R-Squared Value: {r2:.2f}")

## Plotting Regression Line

plt.figure(figsize=(8,5))
plt.scatter(X_test, Y_test, color="blue", label="Actual Data")
plt.plot(X_test, Y_pred, color="red", label="Predicted Trend")
plt.xlabel("Total Immigrants Admission")
plt.ylabel("Total Government Spending")
plt.title("Linear Regression: Immigration vs. Government Spending")
plt.legend()
plt.show()
```

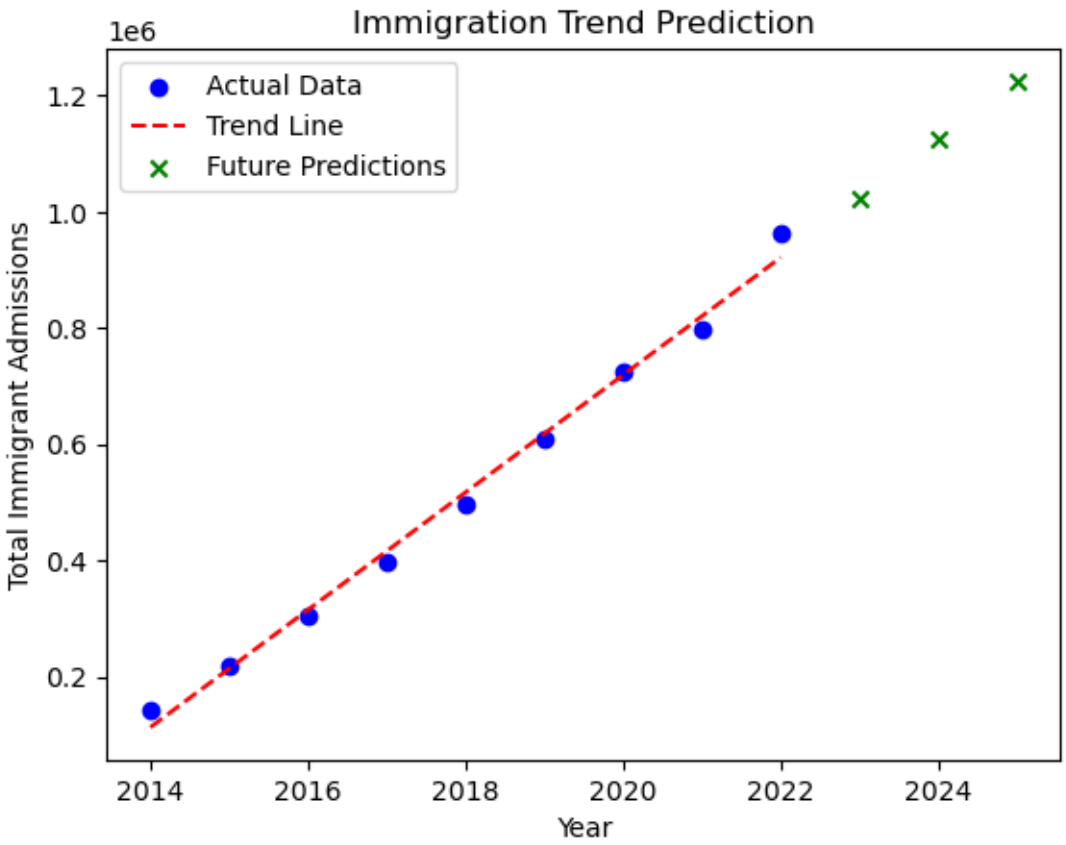
```
Intercept: 117.00297214000179
Coefficient: [6.59119343e-05]
Mean Squared Error: 43.42
R-Squared Value: 0.87
```



[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Sample Data (Replace with your actual dataset)
# Assume 'years' is your independent variable and 'immigrants' is your␣
 ↪dependent variable
data = {'years': [2014,2015,2016,2017,2018,2019,2020,2021,2022], 'immigrants':␣
 ↪[142565,219300,305925,397455,495655,608580,725165,796370,963865
]}
df = pd.DataFrame(data)

# Prepare Data for Modeling
X = df[['years']]  # Independent variable
y = df['immigrants']  # Dependent variable

# Train Linear Regression Model
model = LinearRegression()
model.fit(X, y)

# Predict Future Trends
future_years = np.array([2023,2024,2025]).reshape(-1, 1)  # Extend predictions
```

```
future_predictions = model.predict(future_years)

# Visualize Trends
plt.scatter(df['years'], df['immigrants'], color='blue', label='Actual Data')
plt.plot(df['years'], model.predict(X), color='red', linestyle='--',␣
 ↪label='Trend Line')
plt.scatter(future_years, future_predictions, color='green', marker='x',␣
 ↪label='Future Predictions')

plt.xlabel("Year")
plt.ylabel("Total Immigrant Admissions")
plt.legend()
plt.title("Immigration Trend Prediction")
plt.show()

# Display Future Predictions
future_df = pd.DataFrame({'Year': future_years.flatten(), 'Predicted␣
 ↪Immigrants': future_predictions})
print(future_df)
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X
does not have valid feature names, but LinearRegression was fitted with feature
names
  warnings.warn(
```



	Year	Predicted Immigrants
0	2023	1.022710e+06
1	2024	1.123810e+06
2	2025	1.224911e+06

[ ]:
```
## Storing cleaned datsets

# Creating a folder to save cleaned files
folder_name = "Cleaned_Data"
os.makedirs(folder_name, exist_ok=True)

# Saving Cleaned Files
immigrants_admissions_category.to_excel(os.path.join(folder_name,␣
 ↪"immigrants_admissions_category.xlsx"), index=False)
annual_average_growth.to_excel(os.path.join(folder_name, "annual_average_growth.
 ↪xlsx"), index=False)
immigrants_income_amounts.to_excel(os.path.join(folder_name,␣
 ↪"immigrants_income_amounts.xlsx"), index=False)
immigrants_income_proportions.to_excel(os.path.join(folder_name,␣
 ↪"immigrants_income_proportions.xlsx"), index=False)
```

```
government_expense_per_capita.to_excel(os.path.join(folder_name,␣
 ↪"government_expense_per_capita.xlsx"), index=False)
government_spending_per_sector.to_excel(os.path.join(folder_name,␣
 ↪"government_spending_per_sector.xlsx"), index=False)
social_assistance_case_yearly.to_excel(os.path.join(folder_name,␣
 ↪"social_assistance_case_yearly.xlsx"), index=False)
total_government_spending.to_excel(os.path.join(folder_name,␣
 ↪"total_government_spending.xlsx"), index=False)
total_income_residents.to_excel(os.path.join(folder_name,␣
 ↪"total_income_residents.xlsx"), index=False)
immigrants_income_amounts_with_tax.to_excel(os.path.join(folder_name,␣
 ↪"immigrants_income_amounts_with_tax.xlsx"), index=False)
```